

Week 4: Tabular data

MY472: Data for Data Scientists

https://lse-my472.github.io/

Autumn Term 2026

Ryan Hübert

Associate Professor of Methodology

Slides last updated: 21 October 2026

Outline

1. Types and shapes of data
2. Creating tables in Python
3. Strings in Python
4. Reading and writing tabular data

Shapes of data

When a data scientist works with data, it comes in a "shape"

There are many shapes of data, for example:

- Raw text: unstructured sequences of characters
- Key-value: semi-structured data such as `dict` or JSON
- Hierarchical or tree: nested relationships such as HTML
- Geometric or spatial data: coordinates, shapes

The most common shape of data is **tabular**

- Tabular data is arranged in *tables* with rows and columns
- A **cell** is where one row intersects with one column
- Often called "datasets," "data frames," etc.
- Many data shapes are converted to tabular data for analysis

Units and observations

Units are the entities or subjects being studied

- E.g., individuals, countries, companies

An **observation** is a single "peek" at a unit under specific conditions, such as in a time period

In a **cross-sectional dataset**, units = observations

- Take a slice (cross-section) at a single point in time

In a **hierarchical dataset**, units $\neq$ observations

- Each unit can have multiple observations, e.g. **time series** and **longitudinal (panel) data**

In tabular data, *rows are observations*

Features (variables)

Features are attributes of a unit specified for each observation

These are also called **variables** since their values *vary* depending on the observation

In tabular data, *all columns are features*

- They could be numerical or categorical, or merely identifiers

All tabular datasets should have a **primary key**: a variable which uniquely identifies each observation

- This could be implicit (a combination of two or more variables) or explicit, like a unique ID number

A motivating example: peaches

We'll develop ideas around a hypothetical peach seller in the UK

Consider her sales on one day: kilograms sold and price per kilogram, for each variety

variety	colour	kg_sold	gbp_kg
Donut	Yellow	150	4.40
Nectarine	Red	100	7.75
White	White	120	5.00

Here we have cross-sectional data:

- Mix of data types: categorical, integer, floating point
- Unit and primary key is peach variety (some fuzziness, though)

Outline

1. Types and shapes of data
2. Creating tables in Python
3. Strings in Python
4. Reading and writing tabular data

Tabular data in Python

Python has no tabular object of its own, so we use two packages

- `numpy` has **array** objects
- `pandas` has **data frame** objects

Common to import `numpy` and `pandas` using an alias:

```
import numpy as np
import pandas as pd
```

Numpy arrays

`numpy` stores tabular data in arrays:

- must contain homogenous object types
- can have an arbitrary number of dimensions (table is 2D)

```
mat = np.array([[102, 671, 348], [120, 891, 172]]) # a matrix
print(mat)
```

```
[[102 671 348]
 [120 891 172]]
```

```
print(mat.shape)  # get dimensions
```

```
(2, 3)
```

```
print(mat.dtype)  # type of each element
```

```
int64
```

`dtype` is the *data type*: integer, float, string, etc. – defined in `numpy`

`pandas` data frames

`pandas` stores data in 2D `DataFrame` objects where:

- Each column is a separate named object called a **series**
- Each series can contain a different object type (unlike 2D array)

A series is a column of a data frame, similar to `list` except:

- Elements in a series must have a homogeneous* object type
- Has a label for each element called an **index**

```
kg = pd.Series([150, 100, 120])
print(kg)
```

```
0    150
1    100
2    120
dtype: int64
```

Built on `numpy`, so it inherits many things, like `dtype`

Indexing a pandas series

By default, it has a **range index** (i.e., the integers $0, 1, 2, \ldots$)

- ▷ It might looks like a “row number”, but it is a *label*
- ▷ You can specify a more informative index

```
kg = pd.Series([150, 100, 120],
               index = ["Donut", "Nectarine", "White"])
print(kg)
```

```
Donut        150
Nectarine    100
White        120
dtype: int64
```

You get a value from the series by its label:

```
print(kg["Nectarine"])
```

```
100
```

A series is like a `dict`

A series is like a `dict`, so you can make a series from a `dict`

```
kg = pd.Series({"Donut": 150, "Nectarine": 100, "White": 120})
print(kg["Nectarine"]) # Extract value using key, like a dict!
```

```
100
```

If the values have different types, the dtype is `object`

```
person = pd.Series({"name": "John Smith", "kg_sold": 100})
print(person)
```

```
name        John Smith
kg_sold            100
dtype: object
```

`object` is a catch-all that should be avoided since it signals the type is “unknown”

- ▷ Sometimes useful when reading data into Python (later)

Data frames and series

A `pandas` data frame is a collection of series created from a `dict` where each key is a column name,and each value is a series

```
kg = pd.Series({"Donut": 150, "Nectarine": 100, "White": 120})
price = pd.Series({"Donut": 4.4, "Nectarine": 7.75, "White": 5})
df = pd.DataFrame({"kg_sold": kg, "gbp_kg": price})
print(df)
```

```
           kg_sold  gbp_kg
Donut          150    4.40
Nectarine      100    7.75
White          120    5.00
```

Look at all the column types by using `.dtypes`

```
print(df.dtypes)
```

```
kg_sold      int64
gbp_kg     float64
dtype: object
```

Creating a data frame

Usually, you give `pd.DataFrame()` a `dict` of lists, not series

```
sales = pd.DataFrame({
  "variety": ["Donut", "Nectarine", "White"],
  "colour": ["Yellow", "Red", "White"],
  "kg_sold": [150, 100, 120],
  "gbp_kg": [4.4, 7.75, 5]
})
print(sales)
```

```
     variety  colour  kg_sold  gbp_kg
0      Donut  Yellow      150    4.40
1  Nectarine     Red      100    7.75
2      White   White      120    5.00
```

Since an index was not specified, a range index was added

- ▷ This is probably better: variety is real data not a label

Data frame from a list of `dicts`

You can also give `pd.DataFrame()` a list of `dicts`, one per row

- `pandas` builds data frame using keys as column names

```
rows = [
  {"variety": "Donut", "colour": "Yellow", "kg_sold": 150},
  {"variety": "Nectarine", "kg_sold": 100}
]
print(pd.DataFrame(rows))
```

```
     variety  colour  kg_sold
0      Donut  Yellow      150
1  Nectarine     NaN      100
```

Notice `NaN` — this is a `numpy` missing value (`np.nan`)

- Why? Second row had no colour data!
- `NaN` stands for “not a number” but it’s technically a float

Selecting columns

`.columns` shows the column names

```
print(sales.columns)
```

```
Index(['variety', 'colour', 'kg_sold', 'gbp_kg'], dtype='str')
```

You can change column names using `.rename()`:

```
sales = sales.rename(columns = {"gbp_kg" : "gbp_per_kg"})
print(sales)
```

```
     variety  colour  kg_sold  gbp_per_kg
0      Donut  Yellow      150        4.40
1  Nectarine     Red      100        7.75
2      White   White      120        5.00
```

Selecting columns

If you want to look at a subset of column(s):

- Single brackets give a column as a (named) *series*

```
print(sales["kg_sold"])
```

```
0    150
1    100
2    120
Name: kg_sold, dtype: int64
```

- Double brackets give a *data frame* (inner brackes say which)

```
print(sales[["kg_sold", "gbp_per_kg"]])
```

```
   kg_sold  gbp_per_kg
0      150        4.40
1      100        7.75
2      120        5.00
```

Changing a dtype

`.astype()` copies a column and converts to a new dtype

```
print(sales["kg_sold"].astype(float))
```

```
0    150.0
1    100.0
2    120.0
Name: kg_sold, dtype: float64
```

```
# sales["kg_sold"] = sales["kg_sold"].astype(float) # to keep change
```

Warning: `pandas` also sometimes changes a dtype by itself, e.g. a missing value turns an integer dtype into floats

```
print(pd.Series([150, None, 120]))
```

```
0    150.0
1      NaN
2    120.0
dtype: float64
```

Selecting by label or position

Two ways to select from a data frame, both as `[row, column]`

- `.loc[]` selects by *label*
- `.iloc[]` selects by *position*

With the default index they agree, because label 0 is in position 0

```
print(sales)
```

```
     variety  colour  kg_sold  gbp_per_kg
0      Donut  Yellow      150        4.40
1  Nectarine     Red      100        7.75
2      White   White      120        5.00
```

```
print(sales.loc[0, "variety"])
```

```
Donut
```

```
print(sales.iloc[0, 0])
```

```
Donut
```

Selecting by label or position

They stop agreeing as soon as you remove rows since remaining rows keep their old labels

Create new table `cheap` removing varieties over £6/kg:

```
print(cheap)
```

```
  variety  colour  kg_sold  gbp_per_kg
0   Donut  Yellow      150         4.4
2   White   White      120         5.0
```

```
print(cheap.iloc[1, 0])   # second row
```

```
White
```

```
print(cheap.loc[1])       # row labelled 1: no longer there
```

```
KeyError: 1
```

The index

`pandas` lines up two columns by their *labels*, not their order

```
print(cheap["kg_sold"] + sales["kg_sold"])
```

```
0    300.0
1      NaN
2    240.0
Name: kg_sold, dtype: float64
```

Label 1 is only in `sales`, so its sum is `NaN`

To renumber the labels, use `.reset_index()`

- `drop = True` discards the old labels

```
print(cheap.reset_index(drop = True))
```

```
  variety  colour  kg_sold  gbp_per_kg
0   Donut  Yellow      150         4.4
1   White   White      120         5.0
```

Selecting by label or position

Use a colon to select an entire row/column (returning a series):

```
print(sales.loc[:, "variety"]) # equivalent to `sales["variety"]`
```

```
0        Donut
1    Nectarine
2        White
Name: variety, dtype: str
```

```
print(sales.loc[0, :])
```

```
variety        Donut
colour        Yellow
kg_sold          150
gbp_per_kg       4.4
Name: 0, dtype: object
```

Extract the data as a “simple” Python list:

```
sales.loc[0, :].tolist()
```

```
['Donut', 'Yellow', np.int64(150), np.float64(4.4)]
```

Adding data

Add a column by assigning it

Suppose the peach sales data was from a specific date (1st July 2026), and she wants to indicate this

```
sales["date"] = "2026-07-01"
print(sales)
```

```
     variety  colour  kg_sold  gbp_per_kg        date
0      Donut  Yellow      150        4.40  2026-07-01
1  Nectarine     Red      100        7.75  2026-07-01
2      White   White      120        5.00  2026-07-01
```

If you supply a *single* value, it is copied to every row

Adding data

Add rows with `pd.concat()`, e.g. data from 1st August 2026

```
sales_august = pd.DataFrame({
  "variety": ["Donut", "Nectarine", "White"],
  "colour": ["Yellow", "Red", "White"],
  "kg_sold": [140, 200, 60],
  "gbp_per_kg": [4, 7.5, 5.1],
  "date": "2026-08-01"  # copied to every row
})
sales = pd.concat([sales, sales_august],
                  axis = 0,              # stack rows
                  ignore_index = True)  # new range index
print(sales)
```

```
     variety  colour  kg_sold  gbp_per_kg        date
0      Donut  Yellow      150        4.40  2026-07-01
1  Nectarine     Red      100        7.75  2026-07-01
2      White   White      120        5.00  2026-07-01
3      Donut  Yellow      140        4.00  2026-08-01
4  Nectarine     Red      200        7.50  2026-08-01
5      White   White       60        5.10  2026-08-01
```

In `pandas`, the `axis` option indicates rows (0) or columns (1)

Date and time data

`pandas` converts text to dates with `pd.to_datetime()`

```
sales["date"] = pd.to_datetime(sales["date"])
print(sales["date"])
```

```
0   2026-07-01
1   2026-07-01
2   2026-07-01
3   2026-08-01
4   2026-08-01
5   2026-08-01
Name: date, dtype: datetime64[us]
```

Notice the dtype is: `datetime64[us]` (from `numpy`)

Can also include times:

```
pd.to_datetime(pd.Series(["2026-07-01 1:01 PM"]))
```

```
0   2026-07-01 13:01:00
dtype: datetime64[us]
```

Date and time data

When date format is ambiguous, `pandas` guesses and warns:

```
pd.to_datetime(pd.Series(["1/7/26", "1/8/26"]))
```

```
<string>:1: UserWarning: Could not infer format, so each element will
be parsed individually, falling back to `dateutil`. To ensure parsing
is consistent and as-expected, please specify a format.
0   2026-01-07
1   2026-01-08
dtype: datetime64[us]
```

Get in the habit of specifying format to avoid problems:

```
pd.to_datetime(pd.Series(["1/7/26", "1/8/26"]), format = "%d/%m/%y")
```

```
0   2026-07-01
1   2026-08-01
dtype: datetime64[us]
```

Find formatting codes in the `strftime` docs

Date and time data

`datetime64` is nice because you can do date operations, like:

```
print(sales["date"] + pd.Timedelta(days = 14))
```

```
0   2026-07-15
1   2026-07-15
2   2026-07-15
3   2026-08-15
4   2026-08-15
5   2026-08-15
Name: date, dtype: datetime64[us]
```

You can also do datetime operations using `.dt` accessor, like extracting days of the week:

```
print(sales["date"].dt.day_name())
```

```
0    Wednesday
1    Wednesday
2    Wednesday
3     Saturday
4     Saturday
5     Saturday
Name: date, dtype: str
```

Categorical data

Tabular datasets usually contain categorical data

- Here: `variety` and `colour` are both categorical

Often you want to leave these as is

- Here: `variety` is part of the primary key

But if you want to do statistical analysis, you will need numbers

- One approach: `category` dtype
- A much better approach: create **dummy variables**, one 0/1 column per category

Statistical packages may convert `category` variables to dummies for you, but get in habit of doing it yourself!

Categorical data

```
dummies = pd.get_dummies(sales["colour"], prefix = "colour", dtype = int)
sdum = pd.concat([sales, dummies], axis = 1)  # side by side
sdum = sdum.drop(columns = "colour")  # no longer needed
print(sdum[["variety", "colour_Yellow", "colour_Red", "colour_White"]])
```

```
     variety  colour_Yellow  colour_Red  colour_White
0      Donut              1           0             0
1  Nectarine              0           1             0
2      White              0           0             1
3      Donut              1           0             0
4  Nectarine              0           1             0
5      White              0           0             1
```

```
print(sdum.loc[:, ["variety", "colour_Yellow", "colour_Red", "colour_White"]])
```

```
     variety  colour_Yellow  colour_Red  colour_White
0      Donut              1           0             0
1  Nectarine              0           1             0
2      White              0           0             1
3      Donut              1           0             0
4  Nectarine              0           1             0
5      White              0           0             1
```

Outline

1. Types and shapes of data
2. Creating tables in Python
3. Strings in Python
4. Reading and writing tabular data

Strings in Python

If tables are stored (and read) as text, then you will regularly need string methods for data cleaning

For example, you can

- find patterns, e.g., `in`, `.find()`, `.startswith()`
- "mutate" strings, e.g., `.replace()`, `.lower()`
- combine or split strings, e.g., `+`, `.join()`, `.split()`
- clean strings, e.g. `.strip()`

Note: strings are immutable, so these methods return a *new* string

```
my_text = "  Donut peaches  "
my_text.strip() + " Cool cool cool"
```

```
'Donut peaches Cool cool cool'
```

Regular expressions

Searching for patterns in strings is very common in data science

Regular expressions (usually called “regex”) provide a powerful and flexible tool to search (and replace) text

- Similar to globs, but more powerful

Many editors that work with plain text (e.g. Positron) can usually find and replace terms with regular expressions

Can also be used in many programming languages, e.g. when counting or collecting certain keywords in text analysis

Topic could fill lectures itself, we will cover some basics here

Regular expressions: syntax

Regular expressions can consist of literal characters and metacharacters

- **Literal characters**: usual text
- **Metacharacters**: `^ $ [] () {} * + . ?` etc.

When a meta character shall be treated as usual text in a search, **escape** it with a backslash `\` (unless it is in a set `[]`)

- For example:
  - searching for `.` will select *any* character
  - searching for `\.` will select the period

In Python, write regex patterns as **raw strings**, e.g. `r"\."`, so Python passes the backslash to the regex unchanged

Regular expressions: syntax

Consider: “Ryan Hübert teaches on MY472 and MY481.”

If you wanted to find all the mentions of MY courses in this string, then you could use any one of these regex patterns:

- `MY[0-9][0-9][0-9]`
- `MY[0-9]+`
- `MY.{3}`
- `MY[^A-Za-z]{3}`

Some are better and some are worse, but regex gives *tonnes* of flexibility to find patterns

Regular expressions: syntax

In addition to finding characters and “wildcards”, regex allows for:

- **Booleans**, e.g., find one pattern *or* another
- **Capturing groups**, find a specific group of characters (good for extracting later)

For example:

- `MY(481|472)` finds either MY481 or MY472 (or both)

In this pattern, `(481|472)` contains a boolean but it is also a capturing group

Strings in Python

You can use regex patterns with functions in the built-in `re` module

```
import re
rh = "Ryan Hübert teaches on 472 and 481."
print(rh)
```

```
Ryan Hübert teaches on 472 and 481.
```

```
re.sub(r"([^A-Za-z ]{3})", r"MY\1", rh)
```

```
'Ryan Hübert teaches on MY472 and MY481.'
```

Can do some of this without regex too

- E.g., `in` operator as in this list comprehension

```
my_list = ["Ryan Hübert", "Beyoncé", "Bad Bunny"]
["ü" in x for x in my_list]
```

```
[True, False, False]
```

Cleaning a column

A series is not a string, so `str` methods do not work on it directly

```
sales["variety"].lower()
```

```
AttributeError: 'Series' object has no attribute 'lower'
```

Use `.str` "accessor," which applies a string method to every value

```
print(sales["variety"].str.upper())
```

```
0        DONUT
1    NECTARINE
2        WHITE
3        DONUT
4    NECTARINE
5        WHITE
Name: variety, dtype: str
```

Cleaning a column

Regex methods can be used as well, for example:

```
prices = pd.Series(["£4.40", "£7.75", "5"])
clean = prices.str.replace(r"[^0-9.]", "", regex = True)
print(clean)
```

```
0    4.40
1    7.75
2       5
dtype: str
```

Note: the result is still text, so convert it afterwards:

```
print(clean.astype(float))
```

```
0    4.40
1    7.75
2    5.00
dtype: float64
```

Outline

1 Types and shapes of data

2 Creating tables in Python

3 Strings in Python

4 Reading and writing tabular data

Storing tabular data

Common file formats for storing tabular data:

- Comma-separated values (`.csv`) – ubiquitous and simple
  - Each *line* is an observation
  - Each variable value is separated by a comma
- Application specific (proprietary) formats (.dta, .sav, .xlsx etc.)
  - Can allow for richer representations including meta data
  - More complex, and not necessarily human-readable
  - Can cause problems (lost Covid-19 data)
- Formats designed for large tabular data (`.parquet`)
- (Relational) databases

Often choice is dictated by the source (and size) of the data

Reading and writing tabular data in Python

`pandas` can read many file types directly: `pd.read_csv()`, `pd.read_stata()`, `pd.read_spss()`, `pd.read_excel()`

For example, a `.csv` file:

```
sales = pd.read_csv("data/peach_sales.csv")
print(sales)
```

```
     variety  colour  kg_sold  gbp_per_kg        date
0      Donut  Yellow      150        4.40  2026-07-01
1  Nectarine     Red      100        7.75  2026-07-01
2      White   White      120        5.00  2026-07-01
3      Donut  Yellow      140        4.00  2026-08-01
4  Nectarine     Red      200        7.50  2026-08-01
5      White   White       60        5.10  2026-08-01
```

A `.csv` is a plain text file, so it needs to be decoded on read: `pd.read_csv()` assumes UTF-8, unless otherwise specified

Reading and writing tabular data in Python

For day-to-day storage, standard practice is to write to `.csv` format

```
sales.to_csv("data/peach_sales.csv", index = False, header = True)
```

- `index = False`: don't include index as a separate column
- `header = True`: write column names in first row

Since `.csv` is plain text, you can open it like any other text file:

```
with open("data/peach_sales.csv", mode = "r") as f:
  print(f.read())
```

```
variety,colour,kg_sold,gbp_per_kg,date
Donut,Yellow,150,4.4,2026-07-01
Nectarine,Red,100,7.75,2026-07-01
White,White,120,5.0,2026-07-01
Donut,Yellow,140,4.0,2026-08-01
Nectarine,Red,200,7.5,2026-08-01
White,White,60,5.1,2026-08-01
```

Reading data as text

`pandas` guesses a dtype for each column, and it can guess wrongly

Suppose varieties have product codes, written with leading zeros:

```
codes = pd.DataFrame({"code": ["0041", "0107", "0088"],
                      "variety": ["Donut", "Nectarine", "White"]})
print(codes)
```

```
   code    variety
0  0041      Donut
1  0107  Nectarine
2  0088      White
```

You save and then re-read the data back into Python later:

```
codes.to_csv("data/peach_codes.csv", index = False)
print(pd.read_csv("data/peach_codes.csv")["code"])
```

```
0     41
1    107
2     88
Name: code, dtype: int64
```

Codes are text, but `pandas` read them as integers

Reading data as text

You can specify the dtype when reading a data frame

- `dtype = "object"`: every column is read “as-is” as plain text

```
codes = pd.read_csv("data/peach_codes.csv", dtype = "object")
print(codes["code"])
```

```
0    0041
1    0107
2    0088
Name: code, dtype: object
```

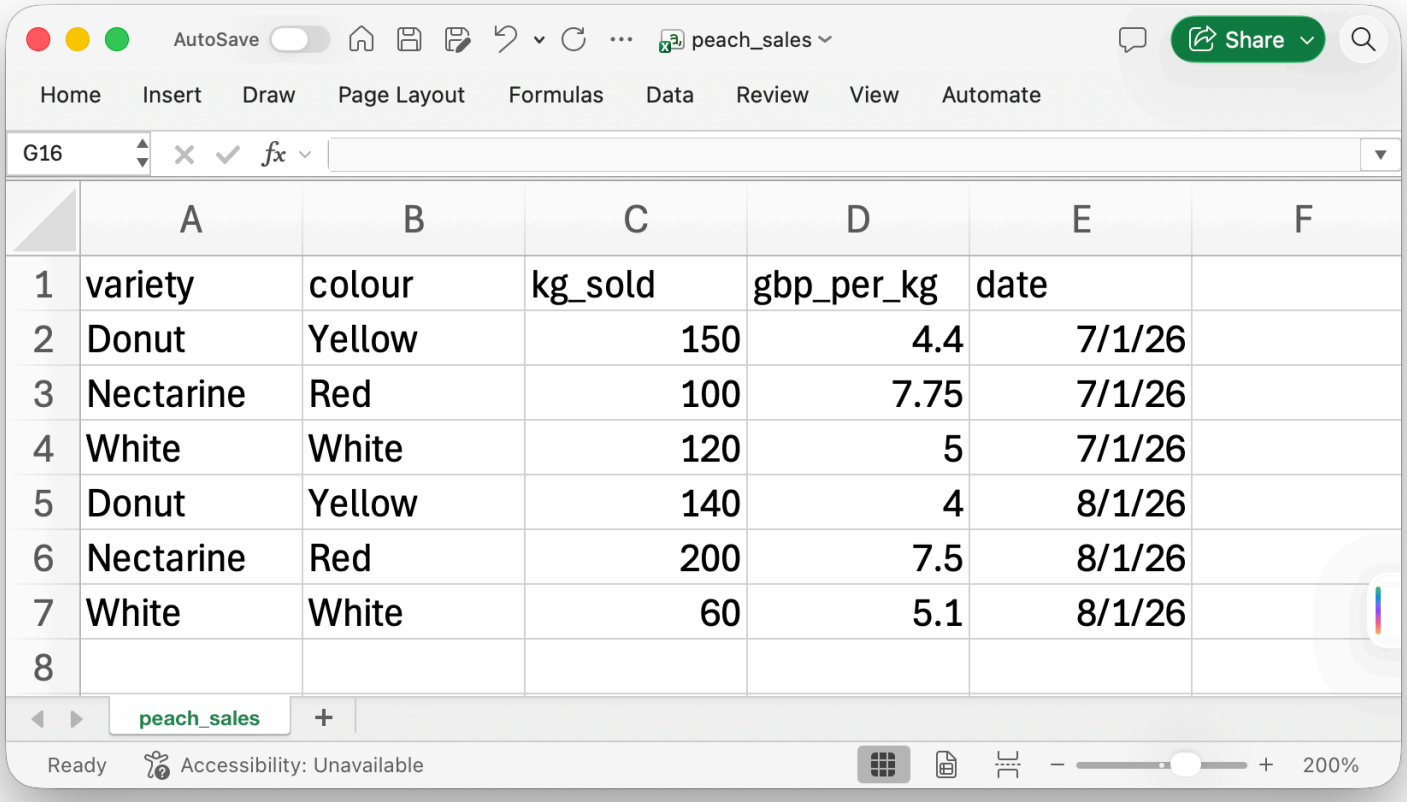
Often the safest way to read an unfamiliar file:

- Nothing is changed when read
- Then convert columns you need afterwards

Can also use `dtype = str` to keep as strings, but “object” is more agnostic and distinguished from *purposeful* `str` columns

Dealing with dates

Suppose the seller opens the `.csv` in Excel in order to look at something, and she sees:



	A	B	C	D	E	F
1	variety	colour	kg_sold	gbp_per_kg	date	
2	Donut	Yellow	150	4.4	7/1/26	
3	Nectarine	Red	100	7.75	7/1/26	
4	White	White	120	5	7/1/26	
5	Donut	Yellow	140	4	8/1/26	
6	Nectarine	Red	200	7.5	8/1/26	
7	White	White	60	5.1	8/1/26	
8						

Dealing with dates

Unbeknownst to her, Excel autosaves the file and next time she reads it in Python, the dates are mangled:

```
sales = pd.read_csv("data/peach_sales.csv")
print(sales["date"])
```

```
0    7/1/26
1    7/1/26
2    7/1/26
3    8/1/26
4    8/1/26
5    8/1/26
Name: date, dtype: str
```

Really bad: can't tell if it's month-day or day-month!

Dealing with dates

She remembers it was July and August, reformats and re-saves

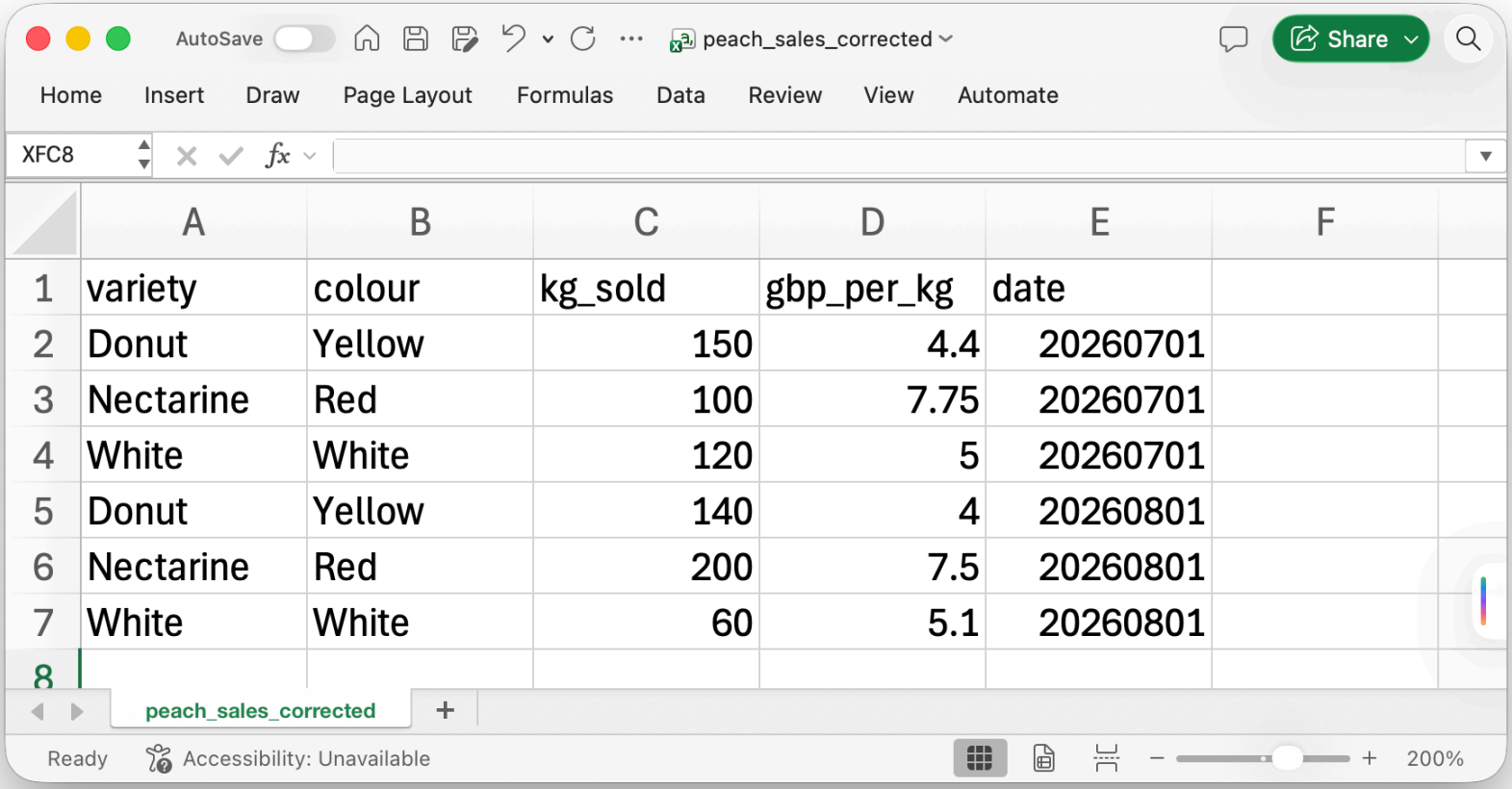
This time, she protects her data from Excel:

```
sales["date"] = pd.to_datetime(sales["date"], format = "%m/%d/%y")
sales["date"] = sales["date"].astype(str).str.replace("-","")
print(sales)
```

```
     variety  colour  kg_sold  gbp_per_kg      date
0      Donut  Yellow      150        4.40  20260701
1  Nectarine     Red      100        7.75  20260701
2      White   White      120        5.00  20260701
3      Donut  Yellow      140        4.00  20260801
4  Nectarine     Red      200        7.50  20260801
5      White   White       60        5.10  20260801
```

```
sales.to_csv("data/peach_sales_corrected.csv", index = False)
```

Dealing with dates



	A	B	C	D	E	F
1	variety	colour	kg_sold	gbp_per_kg	date	
2	Donut	Yellow	150	4.4	20260701	
3	Nectarine	Red	100	7.75	20260701	
4	White	White	120	5	20260701	
5	Donut	Yellow	140	4	20260801	
6	Nectarine	Red	200	7.5	20260801	
7	White	White	60	5.1	20260801	
8						

Dealing with dates

She reads the corrected version into Python and reformats after

```
sales = pd.read_csv("data/peach_sales_corrected.csv")
print(sales)
```

```
     variety  colour  kg_sold  gbp_per_kg      date
0      Donut  Yellow      150        4.40  20260701
1  Nectarine     Red      100        7.75  20260701
2      White   White      120        5.00  20260701
3      Donut  Yellow      140        4.00  20260801
4  Nectarine     Red      200        7.50  20260801
5      White   White       60        5.10  20260801
```

```
sales["date"] = pd.to_datetime(sales["date"], format = "%Y%m%d")
print(sales)
```

```
     variety  colour  kg_sold  gbp_per_kg       date
0      Donut  Yellow      150        4.40 2026-07-01
1  Nectarine     Red      100        7.75 2026-07-01
2      White   White      120        5.00 2026-07-01
3      Donut  Yellow      140        4.00 2026-08-01
4  Nectarine     Red      200        7.50 2026-08-01
5      White   White       60        5.10 2026-08-01
```

Dealing with dates

Dates are challenging but important!

My advice:

- Always try to use ISO 8601 format for dates: YYYY-MM-DD
- Sometimes, even safer if using Excel: YYYYMMDD format
- Read and write to disk as text
- Convert to a date type for analysis only
- Avoid editing `.csv` files in Excel (or other GUIs)

Reading Excel files

An Excel file (`.xlsx`) is not plain text, so `pandas` needs a different function `pd.read_excel()` (requiring `openpyxl`)

An Excel file can contain many **worksheets** (or “sheets”)

- ▷ `pandas` reads one sheet at a time

```
xl = pd.ExcelFile("data/peach_sales.xlsx")
print(xl.sheet_names)                              # list the sheets
```

```
['July', 'August', 'Notes']
```

```
july = pd.read_excel(xl, sheet_name = "July")   # read one sheet
sheets = pd.read_excel(xl, sheet_name = None)   # read all, as dict
```

If you don’t give a `sheet_name`, `pandas` reads only the first sheet

Reading Excel files

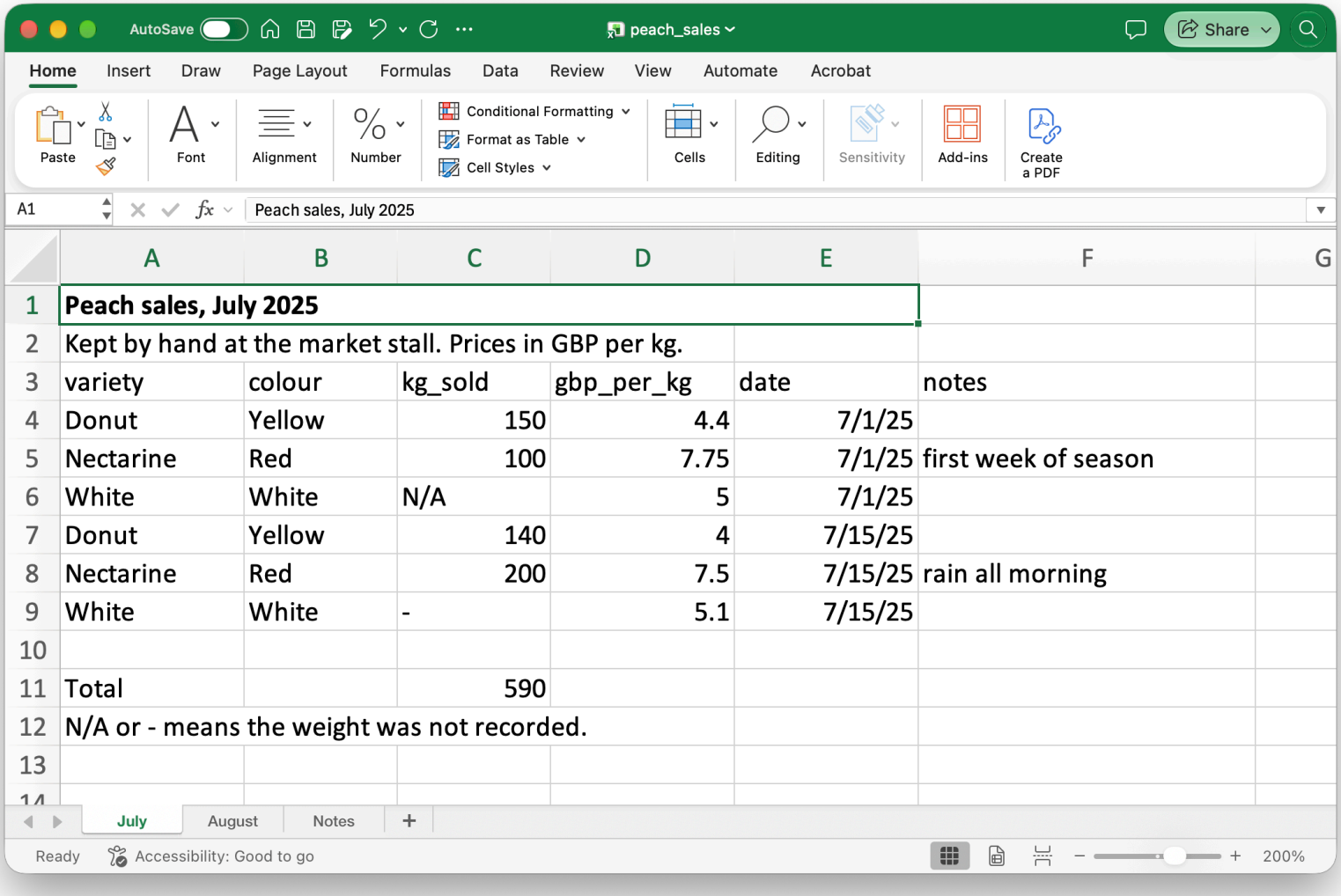
Spreadsheets are often made to be read by people, not by code

- Title rows above the column names
- Notes or totals below the data
- Merged cells: `pandas` reads the value once, then missing values

```
print(july)
```

```
                                Peach sales, July 2025  ...
                                Unnamed: 5
0   Kept by hand at the market stall. Prices in GB...  ...
NaN
1                                              variety  ...
notes
2                                                Donut  ...
NaN
3                                             Nectarine  ...  first
week of season
4                                                White  ...
NaN
5                                                Donut  ...
```

Reading Excel files



	A	B	C	D	E	F	G
1	**Peach sales, July 2025**						
2	Kept by hand at the market stall. Prices in GBP per kg.						
3	variety	colour	kg_sold	gbp_per_kg	date	notes	
4	Donut	Yellow	150	4.4	7/1/25		
5	Nectarine	Red	100	7.75	7/1/25	first week of season	
6	White	White	N/A	5	7/1/25		
7	Donut	Yellow	140	4	7/15/25		
8	Nectarine	Red	200	7.5	7/15/25	rain all morning	
9	White	White	-	5.1	7/15/25		
10							
11	Total		590				
12	N/A or - means the weight was not recorded.						
13							

Reading Excel files

Tell `pandas` which part of the sheet is the table

```
sales = pd.read_excel(xl, sheet_name = "July",
  skiprows = 2,        # skip the two title rows
  usecols = "A:E",     # only columns A to E
  nrows = 6,           # stop before the notes
  na_values = ["N/A", "-"])   # text that means missing
print(sales)
```

```
     variety  colour  kg_sold  gbp_per_kg       date
0      Donut  Yellow    150.0        4.40 2025-07-01
1  Nectarine     Red    100.0        7.75 2025-07-01
2      White   White      NaN        5.00 2025-07-01
3      Donut  Yellow    140.0        4.00 2025-07-15
4  Nectarine     Red    200.0        7.50 2025-07-15
5      White   White      NaN        5.10 2025-07-15
```